```
// foodpanda - script.js
// Works with index.html and style.css (keep all three in the same folder).

const VOUCHERS = { WELCOME: 0.1 }; // WELCOME = 10% off
const EMPTY_CART = { restaurant: "", fee: 0, items: [], voucher: "" };

let cart = { ...EMPTY_CART, items: [] };

try {
  const saved = JSON.parse(localStorage.getItem("foodpanda-cart"));
  if (saved && Array.isArray(saved.items)) cart = { ...EMPTY_CART, ...saved };
} catch (e) {}

const $ = (selector, root = document) => root.querySelector(selector);
const money = (n) => "Rs. " + n.toLocaleString("en-US");

/* ---------- Build the cart drawer ---------- */
document.body.insertAdjacentHTML("beforeend", `
  <div class="cart-overlay"></div>
  <aside class="cart-panel" aria-label="Your cart">
    <div class="cart-head">
      <h2>Your cart</h2>
      <button class="cart-close" aria-label="Close cart">&times;</button>
    </div>
    <p class="cart-restaurant"></p>
    <ul class="cart-items"></ul>
    <p class="cart-empty">Your cart is empty. Add a dish to get started.</p>
    <div class="cart-voucher">
      <input type="text" id="voucher-input" placeholder="Voucher code" aria-
label="Voucher code">
      <button class="btn" id="voucher-apply">Apply</button>
    </div>
    <p class="cart-note" aria-live="polite"></p>
    <div class="cart-pay">
      <label for="pay-method">Pay with</label>
      <select id="pay-method">
        <option>Cash on delivery</option>
        <option>Card</option>
        <option>Mobile wallet</option>
      </select>
    </div>
    <div class="cart-totals">
      <p><span>Subtotal</span><span id="t-subtotal"></span></p>
      <p><span>Delivery fee</span><span id="t-fee"></span></p>
      <p><span>Discount</span><span id="t-discount"></span></p>
      <p class="total"><span>Total</span><span id="t-total"></span></p>
```

```
    </div>
    <button class="btn cart-checkout">Place order</button>
  </aside>
`);

const overlay = $(".cart-overlay");
const panel = $(".cart-panel");
const closeBtn = $(".cart-close");
const list = $(".cart-items");
const emptyMsg = $(".cart-empty");
const restaurantEl = $(".cart-restaurant");
const noteEl = $(".cart-note");
const voucherInput = $("#voucher-input");
const payEl = $("#pay-method");
const checkoutBtn = $(".cart-checkout");
const countEl = $("#cart-count");

/* ---------- Cart logic ---------- */
function save() {
  try {
    localStorage.setItem("foodpanda-cart", JSON.stringify(cart));
  } catch (e) {}
}

function totals() {
  const subtotal = cart.items.reduce((sum, i) => sum + i.price * i.qty, 0);
  const fee = cart.items.length ? cart.fee : 0;
  const rate = VOUCHERS[cart.voucher] || 0;
  const discount = Math.round(subtotal * rate);
  return { subtotal, fee, discount, total: subtotal - discount + fee };
}

function setNote(text, type = "") {
  noteEl.textContent = text;
  noteEl.className = "cart-note " + type;
}

function render() {
  const count = cart.items.reduce((sum, i) => sum + i.qty, 0);
  const t = totals();

  countEl.textContent = count;
  restaurantEl.textContent = cart.items.length ? "From " + cart.restaurant : "";
  emptyMsg.hidden = count > 0;
  checkoutBtn.disabled = count === 0;

  list.innerHTML = cart.items.map((item, index) => `
    <li class="cart-item">
      <div>
        <p class="cart-item-name">${item.name}</p>
```

```
        <p class="cart-item-price">${money(item.price)}</p>
      </div>
      <div class="qty">
        <button data-act="dec" data-i="${index}" aria-label="Decrease
${item.name}">&minus;</button>
        <span>${item.qty}</span>
        <button data-act="inc" data-i="${index}" aria-label="Increase ${item.name}">+
</button>
      </div>
    </li>
  `).join("");

  $("#t-subtotal").textContent = money(t.subtotal);
  $("#t-fee").textContent = money(t.fee);
  $("#t-discount").textContent = t.discount ? "- " + money(t.discount) : money(0);
  $("#t-total").textContent = money(t.total);

  save();
}

function addToCart(button) {
  const card = button.closest(".card");
  const restaurant = $(".shop", card).textContent.split(",")[0].trim();
  const fee = parseInt($(".meta span:last-child", card).textContent.replace(/\D/g,
""), 10) || 0;
  const name = button.dataset.name;
  const price = Number(button.dataset.price);

  // All items in one order must come from one restaurant
  if (cart.items.length && cart.restaurant !== restaurant) {
    const ok = confirm(`Your cart has items from ${cart.restaurant}. Start a new cart
with ${restaurant}?`);
    if (!ok) return;
    cart = { ...EMPTY_CART, items: [] };
  }

  cart.restaurant = restaurant;
  cart.fee = fee;

  const existing = cart.items.find((i) => i.name === name);
  if (existing) existing.qty += 1;
  else cart.items.push({ name, price, qty: 1 });

  setNote("");
  render();

  // Small confirmation on the button
  button.textContent = "Added \u2713";
  setTimeout(() => (button.textContent = "Add to cart"), 900);
}
```

```
/* ---------- Open and close the drawer ---------- */
let lastFocus = null;

function openCart() {
  lastFocus = document.activeElement;
  panel.classList.add("open");
  overlay.classList.add("open");
  document.body.style.overflow = "hidden";
  closeBtn.focus();
}

function closeCart() {
  panel.classList.remove("open");
  overlay.classList.remove("open");
  document.body.style.overflow = "";
  if (lastFocus) lastFocus.focus();
}

/* ---------- Events ---------- */
document.addEventListener("click", (e) => {
  const addBtn = e.target.closest(".add-to-cart");
  if (addBtn) addToCart(addBtn);

  if (e.target.closest(".cart-link")) {
    e.preventDefault();
    openCart();
  }
});

closeBtn.addEventListener("click", closeCart);
overlay.addEventListener("click", closeCart);
document.addEventListener("keydown", (e) => {
  if (e.key === "Escape" && panel.classList.contains("open")) closeCart();
});

list.addEventListener("click", (e) => {
  const btn = e.target.closest("button[data-act]");
  if (!btn) return;

  const item = cart.items[Number(btn.dataset.i)];
  if (btn.dataset.act === "inc") item.qty += 1;
  else item.qty -= 1;

  cart.items = cart.items.filter((i) => i.qty > 0);
  if (!cart.items.length) cart = { ...EMPTY_CART, items: [] };
  render();
});

$("#voucher-apply").addEventListener("click", () => {
```

```
  const code = voucherInput.value.trim().toUpperCase();

  if (!cart.items.length) return setNote("Add a dish before applying a voucher.",
"error");
  if (!code) return setNote("Enter a voucher code.", "error");
  if (!VOUCHERS[code]) return setNote("This voucher code is not valid.", "error");

  cart.voucher = code;
  render();
  setNote(`Voucher ${code} applied: ${VOUCHERS[code] * 100}% off.`, "ok");
});

checkoutBtn.addEventListener("click", () => {
  const t = totals();
  const orderId = "O" + Math.floor(1000 + Math.random() * 9000);
  const message = `Order ${orderId} placed with ${cart.restaurant}. Total
${money(t.total)}, paying by ${payEl.value.toLowerCase()}.`;

  cart = { ...EMPTY_CART, items: [] };
  voucherInput.value = "";
  render();
  setNote(message, "ok");
});

/* ---------- Search ---------- */
const searchBox = $("#search-box");
const cards = document.querySelectorAll(".card");
const restos = document.querySelectorAll(".resto");

$(".restaurants").insertAdjacentHTML(
  "afterend",
  '<p class="empty" hidden>No dishes match your search. Try a different name or
cuisine.</p>'
);
const noResults = $(".empty");

function applySearch() {
  const query = searchBox.value.trim().toLowerCase();

  cards.forEach((card) => {
    const text = [$("h3", card).textContent, $(".shop", card).textContent,
card.dataset.cuisine].join(" ").toLowerCase();
    card.hidden = !text.includes(query);
  });

  restos.forEach((resto) => {
    const text = (resto.textContent).toLowerCase();
    resto.hidden = !text.includes(query);
  });
```

```
  // Show the message if no dish is visible (search and cuisine filter combined)
  noResults.hidden = [...cards].some((card) => card.offsetParent !== null);
}

searchBox.addEventListener("input", applySearch);
document.querySelectorAll(".filter-input").forEach((radio) =>
radio.addEventListener("change", applySearch));

render();
```